

Lion Guardians Vehicles Report

Technical Guide

Vehicle Movement Analysis — Methodology & Calculation Reference
Version 1.1 · Generated July 21, 2026

1. Overview

The **LG Vehicles Report** workflow analyses GPS-tracked vehicle movement in the Amboseli ecosystem for the **Lion Guardians** programme. It ingests telemetry from **EarthRanger** for the *Vehicles* subject group, computes speed maps, vehicle tracks, and a speed-over-time line chart per subject, generates speed and distance statistics, and delivers an interactive dashboard plus a print-ready Word report.

The workflow produces two map visualisations (Speed Map and Vehicle Tracks), a speed line chart, four scalar metric widgets, and a per-subject summary CSV for each tracked vehicle.

Note: The grouper is fixed to `subject_name` — one set of outputs is produced per individual vehicle.

2. Dependencies & Prerequisites

2.1 EarthRanger Connection

All telemetry is fetched from an **EarthRanger** instance via `set_er_connection`. The workflow calls `get_subjectgroup_observations` for the subject group **Vehicles** with filter: `clean`, discarding junk-flagged observations before they reach the pipeline. Subject details and subject-source details are not included in the raw fetch (`include_details: false`).

2.2 Grouper

The grouper is pre-configured to `subject_name` and is not user-selectable. All downstream map, metric, and report tasks iterate over individual vehicles.

2.3 Study Area Layers (from EarthRanger)

The boundary layers here are **not** downloaded from Dropbox — they are fetched live from the connected EarthRanger instance as spatial features via `get_spatial_features` (`select_geo_er`), then composited into a single static layer set by `create_spatial_features_layer` (`spatial_features_layer`). Only two feature types are queried:

Feature type	Style	Grouped by
Conservancies	Filled, sage green #8fbc8b, 75 % fill opacity	<code>type_name</code>
Group Ranch Boundaries	Unfilled, black outline, 1.25 px stroke width	<code>type_name</code>

Note: There is no separate conflict-hotspot layer in the current spec — every subject-level map is composited against exactly these two EarthRanger-sourced layers via `combine_deckgl_map_layers`.

2.4 Word Document Templates & Logo

File	Source	Purpose
<code>vehicles_cover_page.docx</code>	Dropbox	Report cover page template — subject count, time range, preparer
<code>custom_vehicle_template.docx</code>	Dropbox	Per-subject section template — speed map, tracks map, line chart, summary table
<code>lion-guardians.png</code>	Dropbox	Fixed organisation logo used on the cover page — not user-configurable

All three files use `overwrite_existing: false`.

2.5 Base Map Tile Layers

Layer	Opacity	Max zoom
ArcGIS World Hillshade	100 %	15
ArcGIS World Street Map	15 %	15

The hillshade provides full-opacity terrain context. The street map is overlaid at 15 % to show roads and settlement names without obscuring the vehicle data layers.

3. Data Ingestion Pipeline

3.1 Observations → Relocations

`process_relocations` converts raw EarthRanger observations to a standardised GeoDataFrame. Retained columns:

Column	Source field	Description
<code>groupby_col</code>	<code>internal</code>	Subject identifier for grouping
<code>fixtime</code>	<code>observation timestamp</code>	UTC datetime of the GPS fix
<code>junk_status</code>	EarthRanger flag	True if fix is marked junk
<code>geometry</code>	<code>lat/lon</code>	Point geometry (WGS 84)
<code>extra__subject__name</code>	<code>subject.name</code>	Vehicle display name
<code>extra__subject__subject_subtype</code>	<code>subject.subject_subtype</code>	Subject subtype
<code>extra__subject__sex</code>	<code>subject.sex</code>	Subject sex field

Three bad coordinate pairs are removed unconditionally:

- (180.0, 90.0) — boundary sentinel
- (0.0, 0.0) — null-island artefact
- (1.0, 1.0) — common default / test value

Cleaned relocations are persisted as `vehicle_relocations.geoparquet`.

3.2 Relocations → Trajectories

`relocations_to_trajectory` connects consecutive fixes per vehicle into LineString segments, adding `dist_meters`, `speed_kmh`, `segment_start`, and `segment_end`. Trajectories are persisted as `vehicle_trajectories.geoparquet`.

3.3 Temporal Index, Speed Classification & Column Renaming

`add_temporal_index` keys the trajectory GeoDataFrame to `segment_start`, grouped by `subject_name`. `apply_classification` then bins `speed_kmh` into **6 equal-interval classes** (output column: `speed_bins`, labels to 1 decimal place). Three columns are renamed via `map_columns`:

Original column	Renamed to
<code>extra__name</code>	<code>subject_name</code>

Original column	Renamed to
extra__sex	subject_sex
extra__subject_subtype	subject_subtype

The renamed GeoDataFrame is split into per-vehicle partitions by `split_groups`. All downstream map and metric tasks iterate over these partitions via `mapvalues`.

4. Static Map Layers

The two EarthRanger-sourced study area layers (Section 2.3) are built once and composited onto every vehicle-level map to provide spatial context.

4.1 Layer Styles

Layer	Colour	Opacity	Filled	Notes
Conservancies	#8fbc8b sage green	75 %	Yes	Grouped by <code>type_name</code> , legend title “Map Layers”
Group Ranch Boundaries	#000000 black outline	0 % fill	No	Outline only, stroke width 1.25

5. Map Outputs — Methodology

5.1 Speed Map

The speed map renders each trajectory segment coloured by its speed bin. Processing steps before layer creation:

- Sort segments ascending by `speed_bins` so faster segments render on top.
- Apply a custom 6-colour diverging palette to `speed_bins`:

Bin (slowest → fastest)	Hex colour	Visual tone
1 — slowest	#1a9850	Dark green
2	#91cf60	Light green
3	#d9ef8b	Yellow-green
4	#fee08b	Yellow
5	#fc8d59	Orange
6 — fastest	#d73027	Red

- Format bin labels and speed values to 1 decimal place with km/h unit.
- Retain only: `dist_meters`, `speed_bins_colormap`, `geometry`, `speed_kmhr`, `speed_bins_formatted`.

`create_path_layer` renders the filtered GeoDataFrame with width 2.25 px (min 2 px, max 8 px, screen-space), rounded caps and joins, 45 % opacity. Legend title: *Speed (km/h)*. The layer is combined with the two static study area layers. The map is auto-zoomed to the trajectory extent and persisted as HTML (suffix: `speedmap`), then converted to PNG at 2× scale with a 40 s tile-load wait.

5.2 Vehicle Tracks Map

`create_path_layer` renders all trajectory segments per vehicle as a uniform blue path:

Property	Value
Colour	RGB(0, 0, 255) — blue
Width	2.25 px, min 2 px, max 8 px (screen-space)
Cap / Join	Rounded
Opacity	45 %
Max zoom	15 (draw_map setting)

The path layer is combined with the two static study area layers. The view state is shared with the Speed Map (same envelope and zoom). The map is persisted as HTML (suffix: tracks) and converted to PNG at 2× scale with a 40 s tile-load wait.

5.3 Speed Line Chart

`draw_line_chart` plots vehicle speed over the analysis period for each subject individually (no category grouping):

Property	Value
X axis	segment_start — formatted as YYYY-MM-DD, ticks rotated −45°
Y axis	speed_kmphr — labelled Speed (km/h), 1 d.p. ticks
Line shape	Linear (no smoothing)
Background	#f5f5f5 plot area, #e0e0e0 gridlines
Hover mode	x unified
Legend	Hidden

The chart is persisted as HTML (suffix: speed_line_chart) and converted to PNG via `html_to_png` (2× scale, 10 ms wait). Widget title: *Vehicle Speed Over Time*.

6. Summary Metrics

6.1 Per-Vehicle Summary Table

`summarize_df` aggregates trajectory data grouped by `subject_name`:

Output column	Source	Aggregator	Unit	D.p.
mean_speed	speed_kmphr	mean	km/h	2
min_speed	speed_kmphr	min	km/h	2
max_speed	speed_kmphr	max	km/h	2
total_distance	dist_meters	sum	m → km	2

The summary DataFrame (`reset_index: true`) is persisted directly to CSV — one row per vehicle, with no separate totals row appended.

6.2 Scalar Dashboard Widgets

Four scalar values are derived per group by summing the per-subject column from the summary table via `dataframe_column_sum`, then rounded to 2 d.p. by `round_off_values`:

Widget title	Source column	Unit
Mean Speed	<code>mean_speed</code>	km/h
Min Speed	<code>min_speed</code>	km/h
Max Speed	<code>max_speed</code>	km/h
Distance covered	<code>total_distance</code>	km

7. Word Report (.docx)

7.1 Cover Page

`prepare_cover_metadata` (`create_cover_tpl_context`) builds the cover context using the count of unique subjects (`dataframe_column_nunique` on `groupby_col`), the time range, the fixed Lion Guardians logo, and *Ecoscope* as preparer. `create_context_page` (`persist_vehicles_page`) renders it into `cover_page.docx`.

7.2 Per-Vehicle Sections

`ecoscope_workflows_ext_lion_guardians.tasks.reporting.create_vehicles_context` (`build_vehicles_context`) assembles a context dict per vehicle containing the summary table CSV, Speed Map PNG, Tracks Map PNG, and Speed Line Chart PNG. `ecoscope_workflows_ext_lion_guardians.tasks.reporting.render_docx_page` (`create_grouper_doc`) renders each section from the `custom_vehicle_template.docx` template. Image boxes: **11.11 × 6.5 cm**. `strict_images: true` catches missing PNGs before rendering.

7.3 Document Merge

`merge_docx_documents` (`merge_collared_docs`) concatenates the cover page and all per-vehicle sections into a single Word file, saved as `overall_report.docx`.

8. Interactive Dashboard

`gather_dashboard` assembles the vehicles dashboard from seven widget groups:

Widget	Type	Source task
Mean Speed	Single value	<code>round_mean_speed</code> → <code>total_mean_speed_sv_widget</code>
Min Speed	Single value	<code>round_min_speed</code> → <code>total_min_speed_sv_widget</code>
Max Speed	Single value	<code>round_max_speed</code> → <code>total_max_speed_sv_widget</code>
Distance covered	Single value	<code>round_total_distance</code> → <code>total_distance_sv_widget</code>
Speed Map	Map	<code>draw_speedmap</code> → <code>merge_speedmap_widgets</code>
Vehicle Tracks	Map	<code>draw_track_map</code> → <code>merge_tracks_widgets</code>

Widget	Type	Source task
Vehicle Speed Over Time	Line chart	draw_speed_line_chart → grouped_plot_widget_merge

Note: Single-value and chart widget tasks use skipif: [never] so the dashboard always assembles, even when some vehicles have no data.

9. Output Files

All files are written to \$ECOSCOPE_WORKFLOWS_RESULTS.

File / pattern	Format	Content
vehicle_relocations.geoparquet	GeoParquet	Cleaned GPS fix locations
vehicle_trajectories.geoparquet	GeoParquet	Segments with speed_kmh, dist_meters, speed_bins
_speedmap.html	HTML	Interactive speed map per vehicle
_tracks.html	HTML	Interactive tracks map per vehicle
_speed_line_chart.html	HTML	Interactive speed-over-time line chart
_speedmap.png	PNG	2× screenshot of speed map
_tracks.png	PNG	2× screenshot of tracks map
_speed_line_chart.png	PNG	2× screenshot of speed line chart
_summary.csv	CSV	Speed and distance summary table (one row per vehicle)
cover_page.docx	Word	Rendered report cover page
.docx	Word	Per-vehicle report section
overall_report.docx	Word	Final combined Word report

10. Workflow Execution Logic

10.1 Skip Conditions

Two default skip conditions apply to every task (task-instance-defaults):

- **any_is_empty_df** — skips the task (and all dependants) when any input DataFrame is empty, handling vehicles with no fixes gracefully.
- **any_dependency_skipped** — propagates skips downstream automatically.

Single-value widget and chart widget tasks override this with skipif: [never] to ensure the dashboard always assembles.

10.2 Data Flow Summary

Stage	Tasks
Setup	ER connection, time range, grouper (subject_name), base maps
Template download	2 Word templates + 1 logo PNG from Dropbox

Stage	Tasks
Static layers	Conservancies + Group Ranch Boundaries fetched live from EarthRanger
Telemetry ingest	Observations → relocations → trajectories → temporal index → speed bins → rename → split
Speed Map branch	Sort → 6-colour palette → format labels → filter cols → path layer → compose → zoom → HTML → PNG → widget
Tracks branch	Blue path layer → compose → draw map (zoom 15) → HTML → PNG → widget
Line chart branch	draw_line_chart → HTML → PNG → plot widget
Metrics branch	summarize_df → CSV; 4 scalar widgets (mean/min/max speed, distance)
Report assembly	Unique subject count → cover page + per-vehicle sections → merge docx
Dashboard	gather_dashboard combines all 7 widgets

11. Software Versions

As of the migration to the **ecoscope-platform** task/compiler runtime, the workflow's requirements are:

Package	Version	Role
ecoscope-platform	>=2.15.0, <2.16.0	Core task library and workflow engine
ecoscope-workflows-ext-custom	0.1.0rc14.*	Utility tasks (column mapping, screenshots, line chart)
ecoscope-workflows-ext-ste	0.0.0rc1.*	Spatial-features and summary-table tasks
ecoscope-workflows-ext-lion-guardians	0.0.0rc1.*	Lion Guardians Word report rendering tasks
pydeck	0.9.2	Renders the interactive DeckGL maps
opentelemetry-sdk	>=1.20.0, <2.0.0	Workflow run telemetry/observability instrumentation

Note: ecoscope-workflows-core, ecoscope-workflows-ext-ecoscope, ecoscope-workflows-ext-mnc, ecoscope-workflows-ext-icf, and ecoscope-workflows-ext-big-life are not dependencies of this workflow.

Packages are distributed via the prefix.dev conda channel and pinned to compatible versions. The runtime environment is managed by **pixi**.